



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Antispam

Curso: Calidad y Pruebas de Software

Docente: *Patrick Cuadros Quiroga*

Integrantes:

Mamani Cori, Cristhian Carlos

(2023077282)

Jahuira Pilco, Dayan Elvis

(2022075749)

**Tacna – Perú
2026**



Proyecto

Antispam, Tacna, 2026

Presentado por:

Cristhian Mamani Cori
Estudiante de Ingeniería de Sistemas

Dayan Jahaira Pilco
Estudiante de Ingeniería de Sistemas

25/06/2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Cristhian M. - Dayan J.	PCQ	PCQ	29/04/2026	Versión Original

ÍNDICE GENERAL

1. Propuesta narrativa	4
1.1. Planteamiento del Problema	4
1.2. Justificación del proyecto	4
1.3. Objetivo general	4
1.4. Beneficios	4
1.5. Alcance	4
1.6. Requerimientos del sistema	4
1.7. Restricciones	4
1.8. Supuestos	4
1.9. Resultados esperados	4
1.10. Metodología de implementación	4
1.11. Actores claves	4
1.12. Papel y responsabilidades del personal	5
1.13. Plan de monitoreo y evaluación	5
1.14. Cronograma del proyecto	5
1.15. Hitos de entregables	5
2. Presupuesto	5
2.1. Planteamiento de aplicación del presupuesto	5
2.2. Presupuesto	5
2.3. Análisis de Factibilidad	5
2.4. Evaluación Financiera	5

1. Propuesta narrativa

1.1. Planteamiento del Problema

Actualmente, muchas plataformas web carecen de mecanismos eficientes para filtrar comentarios no deseados, lo que requiere horas de revisión manual por parte de moderadores. Los enfoques tradicionales suelen ser vulnerables a variaciones simples en las URLs o palabras clave, y las secciones de comentarios son blanco de ataques masivos de bots que publican enlaces de phishing, publicidad engañosa y contenido irrelevante (Spam). En este contexto, se identifica la necesidad de un sistema robusto de filtrado automático, preparado para entornos de producción modernos en la nube.

1.2. Justificación del proyecto

La inversión en el desarrollo de Aegis Filter se justifica por la eliminación de la carga de moderación manual (con una reducción estimada del 80% del tiempo de revisión), el ahorro estimado de S/ 4,800.00 anuales al evitar la contratación de un moderador humano, y la mejora de la seguridad de la información al bloquear contenido malicioso antes de que llegue a la base de datos de producción. Adicionalmente, el proyecto permite al equipo aplicar prácticas modernas de la industria: Infraestructura como Código (Terraform), contenedores (Docker) e Integración y Despliegue Continuo (CI/CD).

1.3. Objetivo general

Desarrollar e implementar un sistema web seguro y escalable, denominado Aegis Filter, para la gestión y filtrado automático de

comentarios spam, utilizando arquitectura de contenedores e Infraestructura como Código (IaC).

1.4. Beneficios

El desarrollo del proyecto genera los siguientes beneficios:

- Beneficios tangibles: ahorro estimado de S/ 4,800.00 anuales (S/ 400.00 mensuales) al evitar la contratación de un moderador humano.
- Automatización del filtrado de spam, reduciendo en un 80% el tiempo de moderación manual.
- Mejor servicio al cliente y toma acertada de decisiones con métricas en tiempo real.
- Aumento en la confiabilidad de la plataforma y protección de la comunidad frente a phishing y fraudes.
- Fortalecimiento de competencias del equipo en DevOps, nube (Azure) y calidad de software (SonarCloud, Snyk).

1.5. Alcance

El sistema interceptará las peticiones HTTP de creación de comentarios (desarrolladas en Laravel), evaluará el texto utilizando un motor de Expresiones Regulares y Listas Negras (SpamFilterService), y determinará si el registro se inserta en la base de datos MySQL 8 o es rechazado. Dentro del alcance del proyecto se considera:

- Programación del motor de reglas antispam en el backend Laravel 11.
- Aprovisionamiento de infraestructura en la nube (Microsoft Azure) mediante Terraform (IaC).
- Orquestación de la aplicación y la base de datos (MySQL 8) con Docker y Docker Compose.

- Implementación de un flujo completo de Integración y Despliegue Continuo (CI/CD) con GitHub Actions, SonarCloud y Snyk.
- Panel de administración con métricas de comentarios bloqueados y permitidos.

Quedan fuera del alcance el desarrollo de una interfaz de usuario compleja y las pruebas de interfaz (UI/E2E), diferidas para iteraciones posteriores.

1.6. Requerimientos del sistema

Para el desarrollo y ejecución del sistema se requieren los siguientes recursos de hardware y software:

- Computadora personal con procesador Intel i5 / Ryzen 5 y 8/16 GB de RAM.
- Sistema operativo Windows 10/11 con las herramientas de desarrollo (Docker, VS Code).
- PHP 8.2 o superior con el framework Laravel 11 y base de datos MySQL 8.
- Cuenta de Microsoft Azure para la máquina virtual de producción (Standard_B1ms) y Terraform para el aprovisionamiento.
- Repositorio en GitHub con GitHub Actions, SonarCloud y Snyk para el pipeline de calidad.

1.7. Restricciones

El proyecto presenta las siguientes restricciones:

- El código fuente está estrictamente restringido a entornos con PHP 8.2 o superior y Laravel 11.

- La máquina virtual en producción está limitada al plan Standard_B1ms de Azure (1 vCPU, 2 GB de RAM) por presupuesto académico.
- Prohibido el acceso manual FTP al servidor; todo cambio en producción debe realizarse a través del pipeline de GitHub Actions.
- El plazo de desarrollo es de 1 mes (4 semanas) por tratarse de un proyecto académico.

1.8. Supuestos

Para la ejecución del proyecto se asumen las siguientes condiciones:

- Disponibilidad de los servicios de GitHub (Actions, Repositories) y Docker Hub para el funcionamiento del pipeline CI/CD.
- Disponibilidad de créditos académicos de Azure para el alojamiento de la infraestructura.
- El procesamiento de comentarios utiliza datos simulados, cumpliendo normativas básicas de privacidad.

1.9. Resultados esperados

Al finalizar el proyecto se espera obtener los siguientes resultados:

- Una aplicación web basada en Laravel, dockerizada y desplegada en la infraestructura de Microsoft Azure.
- Un filtro automatizado capaz de detectar contenido malicioso, palabras prohibidas y exceso de enlaces antes de persistir en la base de datos.
- Registro de métricas sobre la cantidad de comentarios bloqueados y permitidos.

- Un pipeline de CI/CD operativo con pruebas automatizadas (PHPUnit), análisis de calidad (SonarCloud) y auditoría de seguridad (Snyk).

1.10. Metodología de implementación

El proyecto se desarrolla siguiendo una metodología iterativa e incremental basada en las fases del Proceso Unificado (Inicio, Elaboración, Construcción y Transición), gestionada mediante GitHub Projects (Roadmap) con 4 hitos (Milestones): Lógica Base (Laravel y Servicios), Infraestructura como Código (Terraform en Azure), Contenedorización y CI/CD (Docker y Actions), y Calidad y Seguridad (integración con SonarCloud y Snyk). Cada integración nueva se valida primero contra una instancia local descartable antes de tocar el entorno productivo.

1.11. Actores claves

En la siguiente tabla se presentan los actores claves del proyecto, junto con su descripción y sus responsabilidades principales:

Nombre	Descripción	Responsabilidad
Estudiantes desarrolladores	Equipo responsable del desarrollo del sistema (Cristhian Mamani y Dayan Jahuir)	Diseñar, implementar, probar, desplegar y documentar el sistema según los requisitos establecidos
Administradores y moderadores web	Usuarios finales de la herramienta (administradores de plataformas web y moderadores de contenido)	Utilizar el sistema para gestionar la moderación automática de comentarios y reportar falsos positivos

1.12. Papel y responsabilidades del personal

El equipo de trabajo está conformado por dos estudiantes de Ingeniería de Sistemas, cuyos roles y responsabilidades se detallan a continuación:

Personal	Responsabilidad
Cristhian Mamani Cori	Líder de Proyecto, DevOps y Backend (Terraform, Azure y Laravel)
Dayan Jahaira Pilco	Desarrollador Frontend, QA y Base de Datos (pruebas, Docker y MySQL)

1.13. Plan de monitoreo y evaluación

El avance del proyecto se controla mediante actividades periódicas de monitoreo y evaluación, según se muestra en la siguiente tabla:

	Responsable	Regularmente	Fuentes	Objetivo
Monitoreo	Líder del proyecto	Semanalmente	Reportes del pipeline de CI/CD (GitHub Actions, SonarCloud, Snyk) e informes del equipo.	Proporciona información operativa sobre la calidad y seguridad del código en cada cambio.
Evaluación	Grupo del proyecto	Al cierre de cada hito	Seguimiento de los entregables (FD01 a FD06) y de los hitos del Roadmap.	Verificar el cumplimiento de los objetivos y metas planteados al inicio del proyecto.

1.14. Cronograma del proyecto

El proyecto tiene una duración total de 1 mes (4 semanas), distribuido en las cuatro fases que se detallan a continuación:

Fases	Duración
-------	----------

Inicio	Del 26/05/2026 al 01/06/2026
Elaboración	Del 02/06/2026 al 08/06/2026
Construcción	Del 09/06/2026 al 15/06/2026
Transición	Del 16/06/2026 al 25/06/2026

1.15. Hitos de entregables

Los entregables del proyecto se organizan según las fases del cronograma, tal como se muestra en la siguiente tabla:

	Entregables	Actividades
Inicio	Documento de factibilidad	Viabilidad del proyecto
	Propuesta del proyecto	
	Visión del proyecto	
Elaboración	Documento SRS	Historias de usuario y criterios Gherkin
		Requerimientos funcionales y no funcionales
		Reglas de negocio del filtro
	Documento SAD	Vista lógica (MVC y servicios)
		Diagrama de base de datos (MySQL)
		Diagrama de despliegue (Azure + Docker)
		Diagrama de componentes
	Diccionario de Datos	
	Estándar de programación	
Construcción	Implementación	Código (backend Laravel, bridges y plugin)

Transición	Wiki de GitHub (documentación)
	Pruebas del Sistema (PHPUnit, pytest)

2. Presupuesto

2.1. Planteamiento de aplicación del presupuesto

El presupuesto del proyecto contempla los costos generales de equipamiento, los costos operativos durante el desarrollo, los costos del ambiente en la nube y los costos del personal involucrado, calculados para una duración de 1 mes de desarrollo, conforme al detalle presentado en el Informe de Factibilidad (FD01).

2.2. Presupuesto

El presupuesto total del proyecto asciende a S/ 3,675.00, distribuido en las siguientes categorías:

Categoría	Total (S/.)
Costos Generales	S/. 350.00
Costos Operativos	S/. 200.00
Costos del Ambiente	S/. 125.00
Costos de Personal	S/. 3,000.00
Total	S/. 3,675.00

2.3. Análisis de Factibilidad

2.3.1. Factibilidad Técnica

El proyecto es técnicamente viable: se cuenta con los equipos y herramientas necesarias para su desarrollo, tal como se detalla en la siguiente tabla:

Tipo de Recurso	Nombre	Descripción
Hardware	Equipo	Intel i5 / Ryzen 5
		RAM 8/16 GB
		SSD 1 TB
		Mouse y teclado estándar
Software	Windows 10/11	Sistema Operativo
	Aplicación	PHP 8.2 + Laravel 11
		Docker y Visual Studio Code

2.3.2. *Factibilidad Económica*

La inversión total del proyecto asciende a S/ 3,675.00. Los indicadores financieros calculados en el Informe de Factibilidad (VAN de S/ 4,251.04, TIR de aproximadamente 72.5% y relación Beneficio/Costo de 2.15) confirman que el proyecto es económicamente viable.

2.3.3. *Factibilidad Operativa*

El sistema resulta operativamente viable: automatiza el filtrado de spam reduciendo en un 80% el tiempo de moderación manual, y el equipo cuenta con la capacidad técnica para administrar el sistema mediante Integración Continua (GitHub Actions) y Docker. Adicionalmente, la arquitectura desacoplada del motor antispam permite incorporar nuevos canales de entrada sin modificar el núcleo del sistema.

2.4. Evaluación Financiera

La evaluación financiera del proyecto, considerando un Costo de Oportunidad de Capital (COK) del 12% y una proyección a 3 años con un flujo neto anual de S/ 3,300.00, arroja los siguientes indicadores:

- Valor Actual Neto (VAN): S/ 4,251.04; al ser positivo, se acepta el proyecto.
- Tasa Interna de Retorno (TIR): aproximadamente 72.5%, muy superior al COK del 12%.
- Relación Beneficio/Costo (B/C): 2.15; por cada sol invertido se generan S/ 2.15 en beneficios actualizados.

En consecuencia, todos los indicadores superan los criterios mínimos establecidos, por lo que el proyecto es rentable y financieramente viable.

Anexo 01 – Requerimientos del Sistema *Aegis Filter – Sistema Anti-Spam*¹⁹

RESUMEN EJECUTIVO

Nombre del Proyecto propuesto: <i>Aegis Filter – Sistema Anti-Spam (Antispam), Tacna, 2026</i>	
Propósito del Proyecto y Resultados esperados: El propósito del proyecto es desarrollar e implementar un sistema web seguro y escalable para la gestión y filtrado automático de comentarios spam, utilizando arquitectura de contenedores e Infraestructura como Código (IaC) Los resultados esperados son: • <i>Una aplicación web basada en Laravel, dockerizada y desplegada en Microsoft Azure, capaz de detectar contenido malicioso, palabras prohibidas y exceso de enlaces (Spam), con un pipeline completo de CI/CD (GitHub Actions, SonarCloud y Snyk).</i>	
Población Objetivo: <i>Administradores de plataformas web, moderadores de contenido y comunidades de usuarios finales</i>	
Monto de Inversión (En Soles): <i>S/. 3,675.00</i>	Duración del Proyecto (En Meses): <i>1 mes (4 semanas)</i>